正则表达式

Regular Expression

- 基本正则表达式（BRE,basic regular expression）
- 扩展正则表达式（ERE,extend regular expression）(`grep -E` 生效)

`sed s/[]/g`

`sed -i s/[origin]/[replacement]/g [file_location]`

语法

正则表达式

正则表达式 - 教程
正则表达式 - 简介
正则表达式 - 语法
正则表达式 - 修饰符
正则表达式 - 元字符
正则表达式 - 运算符优先级
正则表达式 - 匹配规则
正则表达式 - 示例
正则表达式 - 在线工具

正则表达式 – 简介

正则表达式 - 修饰符

正则表达式 - 语法

正则表达式(regular expression)描述了一种字符串匹配的模式（pattern），可以用来检查一个串是否含有某种子串、将匹配的子串替换或者从某个串中取出符合某个条件的子串等。

例如：

- `runoo+b`，可以匹配 runoob、runooob、runooooooob 等，`+` 号代表前面的字符必须至少出现一次（1次或多次）。
- `runoo*b`，可以匹配 runob、runoob、runooooooob 等，`*` 号代表前面的字符可以不出现，也可以出现一次或者多次（0次、或1次、或多次）。
- `colou?r` 可以匹配 color 或者 colour，`?` 问号代表前面的字符最多只可以出现一次（0次或1次）。

构造正则表达式的方法和创建数学表达式的方法一样。也就是用多种元字符与运算符可以将小的表达式结合在一起来创建更大的表达式。正则表达式的组件可以是单个的字符、字符集合、字符范围、字符间的选择或者所有这些组件的任意组合。

正则表达式是由普通字符（例如字符 a 到 z）以及特殊字符（称为"元字符"）组成的文字模式。模式描述在搜索文本时要匹配的一个或多个字符串。正则表达式作为一个模板，将某个字符模式与所搜索的字符串进行匹配。

普通字符

普通字符包括没有显式指定为元字符的所有可打印和不可打印字符。这包括所有大写和小写字母、所有数字、所有标点符号和一些其他符号。

字符	描述	实例
`[ABC]`	匹配 `[...]` 中的所有字符，例如 `[aeiou]` 匹配字符串 "google runoob taobao" 中所有的 e o u a 字母。 `/[aeiou]/g` Text Tests google runoob taobao	尝试一下 »
`[^ABC]`	匹配除了 `[...]` 中字符的所有字符，例如 `[^aeiou]` 匹配字符串 "google runoob taobao" 中除了 e o u a 字母的所有字母。 `/[^aeiou]/g` Text Tests google runoob taobao	尝试一下 »
`[A-Z]`	[A-Z] 表示一个区间，匹配所有大写字母，[a-z] 表示所有小写字母。 `/[A-Z]/g` Text Tests Google RunooB Taobao	尝试一下 »
`.`	匹配除换行符（\n、\r）之外的任何单个字符，相等于 [^\n\r]。 `/./g` Text Tests Google RunooB Taobao	尝试一下 »
`[\s\S]`	匹配所有。\s 是匹配所有空白符，包括换行，\S 非空白符，不包括换行。 `/[\s\S]/g` Text Tests Google RunooB Taobao RUnoob taobo	尝试一下 »
`\w`	匹配字母、数字、下划线。等价于 [A-Za-z0-9_] `/\w/g` Text Tests Google RunooB 123Taobao	尝试一下 »

测试工具

修饰符：g

```
[0-9]+
```

匹配文本：

```
123abc456edf789
```

非打印字符

非打印字符也可以是正则表达式的组成部分。下表列出了表示非打印字符的转义序列：

字符	描述
\cx	匹配由x指明的控制字符。例如， \cM 匹配一个 Control-M 或回车符。x 的值必须为 A-Z 或 a-z 之一。否则，将 c 视为一个原义的 'c' 字符。
\f	匹配一个换页符。等价于 \x0c 和 \cL。
\n	匹配一个换行符。等价于 \x0a 和 \cJ。
\r	匹配一个回车符。等价于 \x0d 和 \cM。

分类导航

HTML / CSS
JavaScript
服务端
数据库
数据分析
移动端
XML 教程
ASP.NET
Web Service
开发工具
网站建设

反馈/建议

\s	匹配任何空白字符，包括空格、制表符、换页符等等。等价于 [\f\n\r\t\v]。注意 Unicode 正则表达式会匹配全角空格符。
\S	匹配任何非空白字符。等价于 [^ \f\n\r\t\v]。
\t	匹配一个制表符。等价于 \x09 和 \cI。
\v	匹配一个垂直制表符。等价于 \x0b 和 \cK。

特殊字符

所谓特殊字符，就是一些有特殊含义的字符，如上面说的 `runoo*b` 中的 `*`，简单的说就是表示任何字符串的意思。如果要查找字符串中的 `*` 符号，则需要对 `*` 进行转义，即在其前加一个 `\`，`runo\*ob` 匹配字符串 runo*ob。

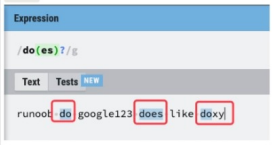
许多元字符要求在试图匹配它们时特别对待。若要匹配这些特殊字符，必须首先使字符"转义"，即，将反斜杠字符 `\` 放在它们前面。下表列出了正则表达式中的特殊字符：

特别字符	描述
$	匹配输入字符串的结尾位置。如果设置了 RegExp 对象的 Multiline 属性，则 $ 也匹配 '\n' 或 '\r'。要匹配 $ 字符本身，请使用 \$。
( )	标记一个子表达式的开始和结束位置。子表达式可以获取供以后使用。要匹配这些字符，请使用 \(和 \)。
*	匹配前面的子表达式零次或多次。要匹配 * 字符，请使用 *。
+	匹配前面的子表达式一次或多次。要匹配 + 字符，请使用 \+。
.	匹配除换行符 \n 之外的任何单字符。要匹配 .，请使用 \ .。
[	标记一个中括号表达式的开始。要匹配 [，请使用 \[。
?	匹配前面的子表达式零次或一次，或指明一个非贪婪限定符。要匹配 ? 字符，请使用 \?。
\	将下一个字符标记为或特殊字符、或原义字符、或向后引用、或八进制转义符。例如，'n' 匹配字符 'n'。'\n' 匹配换行符。序列 '\\' 匹配 "\"，而 '\(' 则匹配 "("。
^	匹配输入字符串的开始位置，除非在方括号表达式中使用，当该符号在方括号表达式中使用时，表示不接受该方括号表达式中的字符集合。要匹配 ^ 字符本身，请使用 \^。
{	标记限定符表达式的开始。要匹配 {，请使用 \{。
|	指明两项之间的一个选择。要匹配 |，请使用 |。

限定符

限定符用来指定正则表达式的一个给定组件必须要出现多少次才能满足匹配。有 `*` 或 `+` 或 `?` 或 `{n}` 或 `{n,}` 或 `{n,m}` 共6种。

正则表达式的限定符有：

字符	描述
*	匹配前面的子表达式零次或多次。例如，`zo*` 能匹配 "z" 以及 "zoo"。`*` 等价于 `{0,}`。
+	匹配前面的子表达式一次或多次。例如，`zo+` 能匹配 "zo" 以及 "zoo"，但不能匹配 "z"。`+` 等价于 `{1,}`。
?	匹配前面的子表达式零次或一次。例如，`do(es)?` 可以匹配 "do" 、 "does"、 "doxy" 中的 "do" 。`?` 等价于 `{0,1}`。 
{n}	n 是一个非负整数。匹配确定的 n 次。例如，`o{2}` 不能匹配 "Bob" 中的 o，但是能匹配 "food" 中的两个 o。
{n,}	n 是一个非负整数。至少匹配n 次。例如，`o{2,}` 不能匹配 "Bob" 中的 o，但能匹配 "fooooood" 中的所有 o。`o{1,}` 等价于 `o+`。`o{0,}` 则等价于 `o*`。
{n,m}	m 和 n 均为非负整数，其中 n <= m。最少匹配 n 次且最多匹配 m 次。例如，`o{1,3}` 将匹配 "fooooood" 中的前三个 o。`o{0,1}` 等价于 `o?`。请注意在逗号和两个数之间不能有空格。

以下正则表达式匹配一个正整数，**[1-9]**设置第一个数字不是 0，`[0-9]*` 表示任意多个数字：

```
/[1-9][0-9]*/
```



请注意，限定符出现在范围表达式之后。因此，它应用于整个范围表达式，在本例中，只指定从 0 到 9 的数字（包括 0 和 9）。

这里不使用 + 限定符，因为在第二个位置或后面的位置不一定需要有一个数字。也不使用 ? 字符，因为使用 ? 会将整数限制到只有两位数。

如果你想设置 0~99 的两位数，可以使用下面的表达式来至少指定一位但至多两位数字。

```
/[0-9]{1,2}/
```

上面的表达式的缺点是，只能匹配两位数字，而且可以匹配 0、00、01、10 99 的章节编号仍只匹配开头两位数字。

改进下，匹配 1~99 的正整数表达式如下：

```
/[1-9][0-9]?/
```

或

```
/[1-9][0-9]{0,1}/
```

`*` 和 `+` 限定符都是贪婪的，因为它们会尽可能多的匹配文字，只有在它们的后面加上一个 `?` 就可以实现非贪婪或最小匹配。

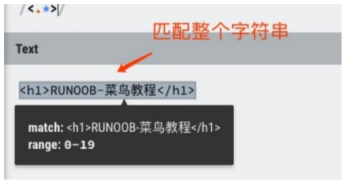
例如，您可能搜索 HTML 文档，以查找在 `h1` 标签内的内容。HTML 代码如下：

```
<h1>RUNOOB-菜鸟教程</h1>
```

贪婪：下面的表达式匹配从开始小于符号 (<) 到关闭 h1 标记的大于符号 (>) 之间的所有内容。

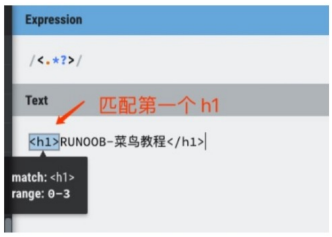
```
/<.*>/
```





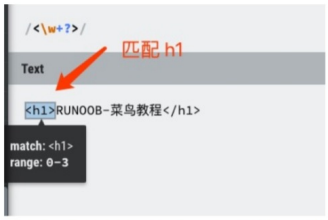
非贪婪：如果您只需要匹配开始和结束 h1 标签，下面的非贪婪表达式只匹配 <h1>。

```
/<.*?>/
```



也可以使用以下正则表达式来匹配 h1 标签，表达式则是：

```
/<\w+?>/
```



通过在 `*`、`+` 或 `?` 限定符之后放置 `?`，该表达式从"贪婪"表达式转换为"非贪婪"表达式或者最小匹配。

定位符

定位符使您能够将正则表达式固定到行首或行尾。它们还使您能够创建这样的正则表达式，这些正则表达式出现在一个单词内、在一个单词的开头或者一个单词的结尾。

定位符用来描述字符串或单词的边界，`^` 和 `$` 分别指字符串的开始与结束，`\b` 描述单词的前或后边界，`\B` 表示非单词边界。

正则表达式的定位符有：

字符	描述
^	匹配输入字符串开始的位置。如果设置了 RegExp 对象的 Multiline 属性，^ 还会与 \n 或 \r 之后的位置匹配。
$	匹配输入字符串结尾的位置。如果设置了 RegExp 对象的 Multiline 属性，$ 还会与 \n 或 \r 之前的位置匹配。
\b	匹配一个单词边界，即字与空格间的位置。
\B	非单词边界匹配。

注意：不能将限定符与定位符一起使用。由于在紧靠换行或者单词边界的前面或后面不能有一个以上位置，因此不允许诸如 `^*` 之类的表达式。

若要匹配一行文本开始处的文本，请在正则表达式的开始使用 `^` 字符。不要将 `^` 的这种用法与中括号表达式内的用法混淆。

若要匹配一行文本的结束处的文本，请在正则表达式的结束处使用 `$` 字符。

若要在搜索章节标题时使用定位点，下面的正则表达式匹配一个章节标题，该标题只包含两个尾随数字，并且出现在行首：

```
/^Chapter [1-9][0-9]{0,1}/
```

真正的章节标题不仅出现行的开始处，而且它还是该行中仅有的文本。它既出现在行首又出现在同一行的结尾。下面的表达式能确保指定的匹配只匹配章节而不匹配交叉引用。通过创建只匹配一行文本的开始和结尾的正则表达式，就可做到这一点。

```
/^Chapter [1-9][0-9]{0,1}$/
```

匹配单词边界稍有不同，但向正则表达式添加了很重要的能力。单词边界是单词和空格之间的位置。非单词边界是任何其他位置。下面的表达式匹配单词 Chapter 的开头三个字符，因为这三个字符出现在单词边界后面：

```
/\bCha/
```

`\b` 字符的位置是非常重要的。如果它位于要匹配的字符串的开始，它在单词的开始处查找匹配项。如果它位于字符串的结尾，它在单词的结尾处查找匹配项。例如，下面的表达式匹配单词 Chapter 中的字符串 ter，因为它出现在单词边界的前面：

```
/ter\b/
```

下面的表达式匹配 Chapter 中的字符串 apt，但不匹配 aptitude 中的字符串 apt：

```
/\Bapt/
```

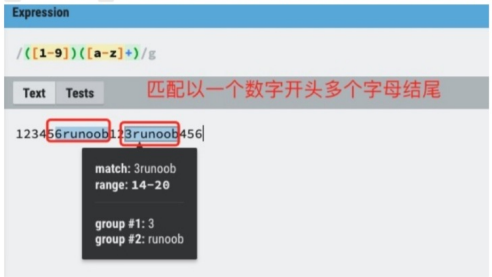
字符串 apt 出现在单词 Chapter 中的非单词边界处，但出现在单词 aptitude 中的单词边界处。对于 `\B` 非单词边界运算符，不可以匹配单词的开头或结尾，如果是下面的表达式，就不匹配 Chapter 中的 Cha：

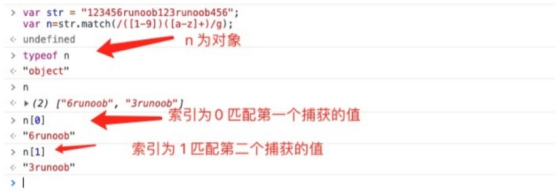
```
\BCha
```

选择

用圆括号 `()` 将所有选择项括起来，相邻的选择项之间用 `|` 分隔。

`()` 表示捕获分组，`()` 会把每个分组里的匹配的值保存起来，多个匹配值可以通过数字 n 来查看(n 是一个数字，表示第 n 个捕获组的内容)。





但用圆括号会有一个副作用，使相关的匹配会被缓存，此时可用 `?:` 放在第一个选项前来消除这种副作用。

其中 `?:` 是非捕获元之一，还有两个非捕获元是 `?=` 和 `?!`，这两个还有更多的含义，前者为正向预查，在任何开始匹配圆括号内的正则表达式模式的位置来匹配搜索字符串，后者为负向预查，在任何开始不匹配该正则表达式模式的位置来匹配搜索字符串。

以下列出 ?=、?<=、?!、?<! 的使用区别

`exp1(?=exp2)`：查找 exp2 前面的 exp1。



`(?<=exp2)exp1`：查找 exp2 后面的 exp1。



`exp1(?!exp2)`：查找后面不是 exp2 的 exp1。



`(?<!exp2)exp1`：查找前面不是 exp2 的 exp1。



> *更多内容可以参考：正则表达式的先行断言(lookahead)和后行断言(lookbehind)*

反向引用

对一个正则表达式模式或部分模式两边添加圆括号将导致相关匹配存储到一个临时缓冲区中，所捕获的每个子匹配都按照在正则表达式模式中从左到右出现的顺序存储。缓冲区编号从 1 开始，最多可存储 99 个捕获的子表达式。每个缓冲区都可以使用 `\n` 访问，其中 n 为一个标识特定缓冲区的一位或两位十进制数。

可以使用非捕获元字符 `?:`、`?=` 或 `?!` 来重写捕获，忽略对相关匹配的保存。

反向引用的最简单的、最有用的应用之一，是提供查找文本中两个相同的相邻单词的匹配项的能力。以下面的句子为例：

```
Is is the cost of of gasoline going up up?
```

上面的句子很显然有多个重复的单词。如果能设计一种方法定位该句子，而不必查找每个单词的重复出现，那该有多好。下面的正则表达式使用单个子表达式来实现这一点：

实例

查找重复的单词：

```
var str = "Is is the cost of of gasoline going up up";
var patt1 = /\b([a-z]+) \1\b/igm;
document.write(str.match(patt1));
```

尝试一下 »

捕获的表达式，正如 `[a-z]+` 指定的，包括一个或多个字母。正则表达式的第二部分是对以前捕获的子匹配项的引用，即，单词的第二个匹配项正好由括号表达式匹配。`\1` 指定第一个子匹配项。

单词边界元字符确保只检测整个单词。否则，诸如 "is issued" 或 "this is" 之类的词组将不能正确地被此表达式识别。

正则表达式后面的全局标记 `g` 指定将该表达式应用到输入字符串中能够查找到的尽可能多的匹配。

表达式的结尾处的不区分大小写 `i` 标记指定不区分大小写。

多行标记 `m` 指定换行符的两边可能出现潜在的匹配。

反向引用还可以将通用资源指示符 (URI) 分解为其组件。假定您想将下面的 URI 分解为协议（ftp、http 等等）、域地址和页/路径：

```
https://www.runoob.com:80/html/html-tutorial.html
```



普通字符

- `[ABC]` 匹配中括号内所有字符
- `[^ABC]` 匹配除了中括号内的所有字符
- `[A-Z]` 是一个区间，匹配A-Z间的所有字符
- `.` 匹配除换行符之外单个字符
- `[\s\S]` \s匹配所有空白符，包含换行；\S匹配所有非空白符，不包含换行
- `[\w]` 匹配所有字母，数字，下滑线。等同于 `[A-Z a-z 0-9 _]`

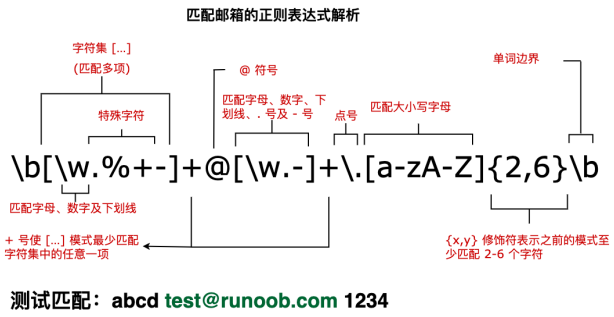
特殊字符（指定匹配次数，出现多少次才可匹配）

自身原本字符的匹配需要转义

- `*` 前面匹配的表达式需要出现0次或多次。eg: `zo*` 可匹配 `z`或`zoo`
- `+` 前面匹配的表达式需要出现1次或多次。eg: `zo+` 可匹配 `zo`或`zoo`
- `?` 前面匹配的表达式需要出现0次或1次

修饰符

`sed s/[]/[]/g<-`

修饰符	含义	描述
i	ignore-不区分大小写	匹配时不区分大小写
g	global-全局匹配	全局匹配
m	multi line-多行匹配	多行匹配
s	特殊字符圆点 . 中包含换行符 `\n`	默认情况下的圆点 **.** 是匹配除换行符 `\n` 之外的任何字符，加上 **s** 修饰符之后，`.` 中包含换行符 `\n`
		

元字符

字符	描述
\	将下一个字符标记为一个特殊字符、或一个原义字符、或一个 向后引用、或一个八进制转义符。例如，'n' 匹配字符 "n"。'\n' 匹配一个换行符。序列 '\\' 匹配 "\" 而 "\(" 则匹配 "("。
^	匹配输入字符串的开始位置。如果设置了 RegExp 对象的 Multiline 属性，^ 也匹配 '\n' 或 '\r' 之后的位置。
$	匹配输入字符串的结束位置。如果设置了RegExp 对象的 Multiline 属性，$ 也匹配 '\n' 或 '\r' 之前的位置。
*	匹配前面的子表达式零次或多次。例如，zo* 能匹配 "z" 以及 "zoo"。* 等价于{0,}。
+	匹配前面的子表达式一次或多次。例如，'zo+' 能匹配 "zo" 以及 "zoo"，但不能匹配 "z"。+ 等价于 {1,}。
?	匹配前面的子表达式零次或一次。例如，"do(es)?" 可以匹配 "do" 或 "does" 。? 等价于 {0,1}。
{n}	n 是一个非负整数。匹配确定的 n 次。例如，'o{2}' 不能匹配 "Bob" 中的 'o'，但是能匹配 "food" 中的两个 o。
{n,}	n 是一个非负整数。至少匹配n 次。例如，'o{2,}' 不能匹配 "Bob" 中的 'o'，但能匹配 "foooood" 中的所有 o。'o{1,}' 等价于 'o+'。'o{0,}' 则等价于 'o*'。
{n,m}	m 和 n 均为非负整数，其中n <= m。最少匹配 n 次且最多匹配 m 次。例如，"o{1,3}" 将匹配 "fooooood" 中的前三个 o。'o{0,1}' 等价于 'o?'。请注意在逗号和两个数之间不能有空格。
?	当该字符紧跟在任何一个其他限制符 (*, +, ?, {n}, {n,}, {n,m}) 后面时，匹配模式是非贪婪的。非贪婪模式尽可能少的匹配所搜索的字符串，而默认的贪婪模式则尽可能多的匹配所搜索的字符串。例如，对于字符串 "oooo"，'o+?' 将匹配单个 "o"，而 'o+' 将匹配所有 'o'。
.	匹配除换行符（\n、\r）之外的任何单个字符。要匹配包括 '\n' 在内的任何字符，请使用像"**(.|\n)**"的模式。
(pattern)	匹配 pattern 并获取这一匹配。所获取的匹配可以从产生的 Matches 集合得到，在VBScript 中使用 SubMatches 集合，在JScript 中则使用 $0…$9 属性。要匹配圆括号字符，请使用 '\(' 或 '\)'。
(?:pattern)	匹配 pattern 但不获取匹配结果，也就是说这是一个非获取匹配，不进行存储供以后使用。这在使用 "或" 字符 (|) 来组合一个模式的各个部分是很有用。例如， 'industr(?:y|ies) 就是一个比 'industry|industries' 更简略的表达式。
(?=pattern)	正向肯定预查（look ahead positive assert），在任何匹配pattern的字符串开始处匹配查找字符串。这是一个非获取匹配，也就是说，该匹配不需要获取供以后使用。例如，"Windows(?=95|98|NT|2000)"能匹配"Windows2000"中的"Windows"，但不能匹配"Windows3.1"中的"Windows"。预查不消耗字符，也就是说，在一个匹配发生后，在最后一次匹配之后立即开始下一次匹配的搜索，而不是从包含预查的字符之后开始。
(?!pattern)	正向否定预查(negative assert)，在任何不匹配pattern的字符串开始处匹配查找字符串。这是一个非获取匹配，也就是说，该匹配不需要获取供以后使用。例如"Windows(?!95|98|NT|2000)"能匹配"Windows3.1"中的"Windows"，但不能匹配"Windows2000"中的"Windows"。预查不消耗字符，也就是说，在一个匹配发生后，在最后一次匹配之后立即开始下一次匹配的搜索，而不是从包含预查的字符之后开始。
(?<=pattern)	反向(look behind)肯定预查，与正向肯定预查类似，只是方向相反。例如，"(?<=95|98|NT|2000)Windows"能匹配"2000Windows"中的"Windows"，但不能匹配"3.1Windows"中的"Windows"。
(?<!pattern)	反向否定预查，与正向否定预查类似，只是方向相反。例如"(?<!95|98|NT|2000)Windows"能匹配"3.1Windows"中的"Windows"，但不能匹配"2000Windows"中的"Windows"。
x|y	匹配 x 或 y。例如，'z|food' 能匹配 "z" 或 "food"。'(z|f)ood' 则匹配 "zood" 或 "food"。
[xyz]	字符集合。匹配所包含的任意一个字符。例如， '[abc]' 可以匹配 "plain" 中的 'a'。
[^xyz]	负值字符集合。匹配未包含的任意字符。例如， '[^abc]' 可以匹配 "plain" 中的'p'、'l'、'i'、'n'。
[a-z]	字符范围。匹配指定范围内的任意字符。例如，'[a-z]' 可以匹配 'a' 到 'z' 范围内的任意小写字母字符。
[^a-z]	负值字符范围。匹配任何不在指定范围内的任意字符。例如，'[^a-z]' 可以匹配任何不在 'a' 到 'z' 范围内的任意字符。
\b	匹配一个单词边界，也就是指单词和空格间的位置。例如， 'er\b' 可以匹配"never" 中的 'er'，但不能匹配 "verb" 中的 'er'。
\B	匹配非单词边界。'er\B' 能匹配 "verb" 中的 'er'，但不能匹配 "never" 中的 'er'。

\cx	匹配由 x 指明的控制字符。例如， \cM 匹配一个 Control-M 或回车符。x 的值必须为 A-Z 或 a-z 之一。否则，将 c 视为一个原义的 'c' 字符。
\d	匹配一个数字字符。等价于 [0-9]。
\D	匹配一个非数字字符。等价于 [^0-9]。
\f	匹配一个换页符。等价于 \x0c 和 \cL。
\n	匹配一个换行符。等价于 \x0a 和 \cJ。
\r	匹配一个回车符。等价于 \x0d 和 \cM。
\s	匹配任何空白字符，包括空格、制表符、换页符等等。等价于 [\f\n\r\t\v]。
\S	匹配任何非空白字符。等价于 [^ \f\n\r\t\v]。
\t	匹配一个制表符。等价于 \x09 和 \cI。
\v	匹配一个垂直制表符。等价于 \x0b 和 \cK。
\w	匹配字母、数字、下划线。等价于'[A-Za-z0-9_]'。
\W	匹配非字母、数字、下划线。等价于 '[^A-Za-z0-9_]'。
\xn	匹配 n，其中 n 为十六进制转义值。十六进制转义值必须为确定的两个数字长。例如，'\x41' 匹配 "A"。'\x041' 则等价于 '\x04' & "1"。正则表达式中可以使用 ASCII 编码。
\num	匹配 num，其中 num 是一个正整数。对所获取的匹配的引用。例如，'(.)\1' 匹配两个连续的相同字符。
\n	标识一个八进制转义值或一个向后引用。如果 \n 之前至少 n 个获取的子表达式，则 n 为向后引用。否则，如果 n 为八进制数字 (0-7)，则 n 为一个八进制转义值。
\nm	标识一个八进制转义值或一个向后引用。如果 \nm 之前至少有 nm 个获得子表达式，则 nm 为向后引用。如果 \nm 之前至少有 n 个获取，则 n 为一个后跟文字 m 的向后引用。如果前面的条件都不满足，若 n 和 m 均为八进制数字 (0-7)，则 \nm 将匹配八进制转义值 nm。
\nml	如果 n 为八进制数字 (0-3)，且 m 和 l 均为八进制数字 (0-7)，则匹配八进制转义值 nml。
\un	匹配 n，其中 n 是一个用四个十六进制数字表示的 Unicode 字符。例如， \u00A9 匹配版权符号 (?)。

运算符优先级

运算符	描述
\	转义符
(), (?:), (?=), []	圆括号和方括号
*, +, ?, {n}, {n,}, {n,m}	限定符
^, $, \任何元字符、任何字符	定位点和序列（即：位置和顺序）
|	替换，"或"操作 字符具有高于替换运算符的优先级，使得"m|food"匹配"m"或"food"。若要匹配"mood"或"food"，请使用括号创建子表达式，从而产生"(m|f)ood"。

匹配规则

正则表达式 - 匹配规则

基本模式匹配

一切从最基本的开始。模式，是正则表达式最基本的元素，它们是一组描述字符串特征的字符。模式可以很简单，由普通的字符串组成，也可以非常复杂，往往用特殊的字符表示一个范围内的字符、重复出现，或表示上下文。例如：

```
^once
```

这个模式包含一个特殊的字符 `^`，表示该模式只匹配那些以 **once** 开头的字符串。例如该模式与字符串 **"once upon a time"** 匹配，与 **"There once was a man from NewYork"** 不匹配。正如如 `^` 符号表示开头一样，`$` 符号用来匹配那些以给定模式结尾的字符串。

```
bucket$
```

这个模式与 **"Who kept all of this cash in a bucket"** 匹配，与 **"buckets"** 不匹配。字符 `^` 和 `$` 同时使用时，表示精确匹配（字符串与模式一样）。例如：

```
^bucket$
```

只匹配字符串 **"bucket"**。如果一个模式不包括 `^` 和 `$`，那么它与任何包含该模式的字符串匹配。例如模式：

```
once
```

与字符串

```
There once was a man from NewYork
Who kept all of his cash in a bucket.
```

是匹配的。
在该模式中的字母 **(o-n-c-e)** 是字面的字符，也就是说，他们表示该字母本身，数字也是一样的。其他一些稍微复杂的字符，如标点符号和白字符（空格、制表符等），要用到转义序列。所有的转义序列都用反斜杠 `\` 打头。制表符的转义序列是 `\t`。所以如果我们要检测一个字符串是否以制表符开头，可以用这个模式：

```
^\t
```

类似的，用 `\n` 表示**"新行"**，`\r` 表示回车。其他的特殊符号，可以用在前面加上反斜杠，如反斜杠本身用 `\\` 表示，句号 `.` 用 `\.` 表示，以此类推。

字符簇

在 INTERNET 的程序中，正则表达式通常用来验证用户的输入。当用户提交一个 FORM 以后，要判断输入的电话号码、地址、EMAIL 地址、信用卡号码等是否有效，用普通的基于字面的字符是不够的。
所以要用一种更自由的描述我们要的模式的办法，它就是字符簇。要建立一个表示所有元音字符的字符簇，就把所有的元音字符放在一个方括号里：

```
[AaEeIiOoUu]
```

这个模式与任何元音字符匹配，但只能表示一个字符。用连字号可以表示一个字符的范围，如：

```
[a-z] // 匹配所有的小写字母
[A-Z] // 匹配所有的大写字母
[a-zA-Z] // 匹配所有的字母
[0-9] // 匹配所有的数字
[0-9\.\-] // 匹配所有的数字，句号和减号
[ \f\r\t\n] // 匹配所有的白字符
```

同样的，这些也只表示一个字符，这是一个非常重要的。如果要匹配一个由一个小写字母和一位数字组成的字符串，比如 "z2"、"t6" 或 "g7"，但不是 "ab2"、"r2d3" 或 "b52" 的话，用这个模式：

```
^[a-z][0-9]$
```

尽管 `[a-z]` 代表 26 个字母的范围，但在这里它只能与第一个字符是小写字母的字符串匹配。
前面曾经提到^表示字符串的开头，但它还有另外一个含义。当在一组方括号里使用 `^` 时，它表示**"非"**或**"排除"**的意思，常常用来剔除某个字符。

还用前面的例子，我们要求第一个字符不能是数字：

```
^[^0-9][0-9]$
```

这个模式与 "&5"、"g7"及"-2" 是匹配的，但与 "12"、"66" 是不匹配的。下面是几个排除特定字符的例子：

```
[^a-z] //除了小写字母以外的所有字符
[^\\\/\^] //除了(\)(/)(^)之外的所有字符
[^\"\'] //除了双引号(")和单引号(')之外的所有字符
```

特殊字符 `.`(点，句号)在正则表达式中用来表示除了"新行"之外的所有字符。所以模式 `^.5$` 与任何两个字符的、以数字5结尾和以其他非"新行"字符开头的字符串匹配。模式 `.` 可以匹配任何字符串，**换行符（\n、\r）除外**。

PHP的正则表达式有一些内置的通用字符簇，列表如下：

字符簇	描述
[[:alpha:]]	任何字母
[[:digit:]]	任何数字
[[:alnum:]]	任何字母和数字
[[:space:]]	任何空白字符
[[:upper:]]	任何大写字母
[[:lower:]]	任何小写字母
[[:punct:]]	任何标点符号
[[:xdigit:]]	任何16进制的数字，相当于[0-9a-fA-F]

确定重复出现

到现在为止，你已经知道如何去匹配一个字母或数字，但更多的情况下，可能要匹配一个单词或一组数字。一个单词有若干个字母组成，一组数字有若干个单数组成。跟在字符或字符簇后面的花括号({})用来确定前面的内容的重复出现的次数。

字符簇	描述
^[a-zA-Z_]$	所有的字母和下划线
^[[:alpha:]]{3}$	所有的3个字母的单词
^a$	字母a
^a{4}$	aaaa
^a{2,4}$	aa,aaa或aaaa
^a{1,3}$	a,aa或aaa
^a{2,}$	包含多于两个a的字符串
^a{2,}	如：aardvark和aaab，但apple不行
a{2,}	如：baad和aaa，但Nantucket不行
\t{2}	两个制表符
.{2}	所有的两个字符

这些例子描述了花括号的三种不同的用法。一个数字 `{x}` 的意思是**前面的字符或字符簇只出现x次**；一个数字加逗号 `{x,}` 的意思是**前面的内容出现x或更多的次数**；两个数字用逗号分隔的数字 `{x,y}` 表示 **前面的内容至少出现x次，但不超过y次**。我们可以把模式扩展到更多的单词或数字：

```
^[a-zA-Z0-9_]{1,}$      // 所有包含一个以上的字母、数字或下划线的字符串
^[1-9][0-9]{0,}$        // 所有的正整数
^\-{0,1}[0-9]{1,}$      // 所有的整数
^[-]?[0-9]+\.?[0-9]+$   // 所有的浮点数
```

最后一个例子不太好理解，是吗？这么看吧：以一个可选的负号（`[-]?`）开头（`^`）、跟着1个或更多的数字(`[0-9]+`)、和一个小数点(`\.`)再跟上1个或多个数字(`([0-9]+`)，并且后面没有其他任何东西(`$`)。下面你将知道能够使用的更为简单的方法。

特殊字符 `?` 与 `{0,1}` 是相等的，它们都代表着： **0个或1个前面的内容** 或 **前面的内容是可选的** 。所以刚才的例子可以简化为：

```
^\-?[0-9]{1,}\.?[0-9]{1,}$
```

特殊字符 `*` 与 `{0,}` 是相等的，它们都代表着 **0 个或多个前面的内容** 。最后，字符 `+` 与 `{1,}` 是相等的，表示 **1 个或多个前面的内容** ，所以上面的4个例子可以写成：

```
^[a-zA-Z0-9_]+$       // 所有包含一个以上的字母、数字或下划线的字符串
^[1-9][0-9]*$         // 所有的正整数
^\-?[0-9]+$           // 所有的整数
^[-]?[0-9]+(\.[0-9]+)?$ // 所有的浮点数
```

当然这并不能从技术上降低正则表达式的复杂性，但可以使它们更容易阅读。

BRE

符号	作用
^	行首
$	行末
^$	组合符号，匹配空行
.	匹配任意字符，不包括空行
\	转义字符
*	匹配的前一个字符出现0次或多次
.*	组合符，匹配所有内容
^.*	组合符，匹配任意多个字符开头的内容
.*$	组合符，匹配任意多个字符结尾的内容
[abc]	匹配[]集合内的任意一个字符，样例也可以写[a-c]
[^abc]	匹配除了^后面的任意字符

ERE

`grep -E` 生效

符号	作用
+	匹配前一个字符一次或多次
[:/]+	匹配集合内符号:或/ 一次或多次
?	匹配前一个字符0次或1次
|	或，同时过滤多个字符串
()	分组过滤，括号内是一个整体
a{n,m}	匹配前一个字符最少n次，最大m次
a{n,}	匹配前一个字符最少n次
a{n}	匹配前一个字符正好n次
a{,m}	匹配前一个字符最多m次

Linux三剑客

grep

sed

`"2,3p"`打印第2,3行 `"2,+3p"`从第二行往下打印3行

默认工作流输出文件全部，`-n`可以取消

`-i`写入文件

awk